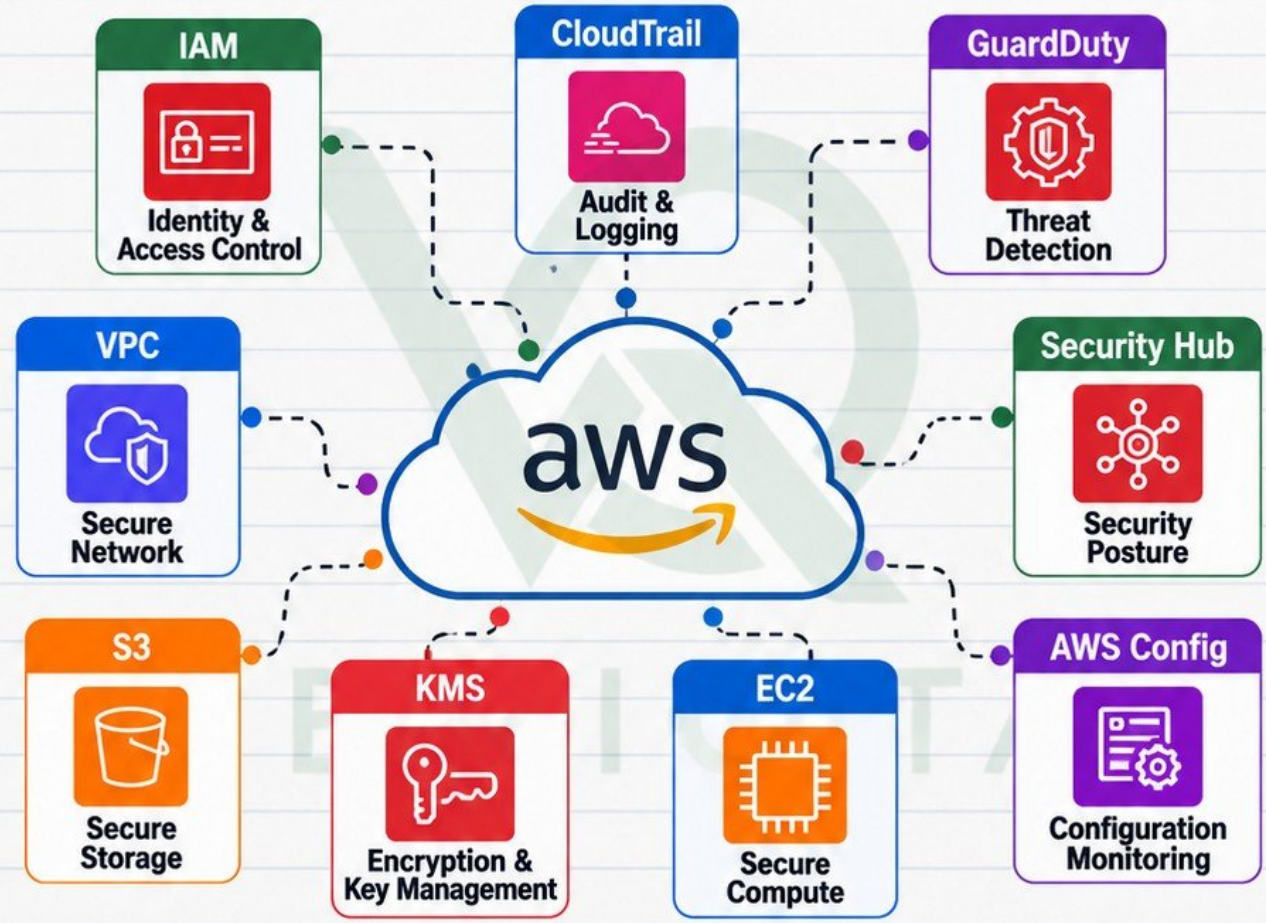
VERIQTA

AWS Cloud Security for Beginners

Secure Your First AWS Environment Step by Step



Real-World Examples | Hands-On Practice | Security Best Practices | Troubleshooting & Verification

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

01 Your First AWS Security Assignment

VERIQTA

1. The Scenario

You have joined a small company as a junior cloud engineer. The company wants to launch a web application on AWS and has asked you to help secure the AWS environment from the beginning.



2. What You Will Learn

- What the shared responsibility model means
- The difference between securing the AWS account and securing resources
- Key assets in a typical AWS environment
- Real risks and common beginner mistakes
- Your first security checklist

3. Shared Responsibility Model

AWS Is Responsible For

- Physical data centers
- Networking infrastructure
- Hardware and virtualization
- Global infrastructure
- Availability of AWS services

aws SECURITY OF THE CLOUD

You Are Responsibile For

- Securing your AWS account
- Managing identities and access
- Configuring and protecting resources
- Data encryption and backups
- Monitoring, logging and incident response

SECURITY IN THE CLOUD

4. Account vs Resource Security

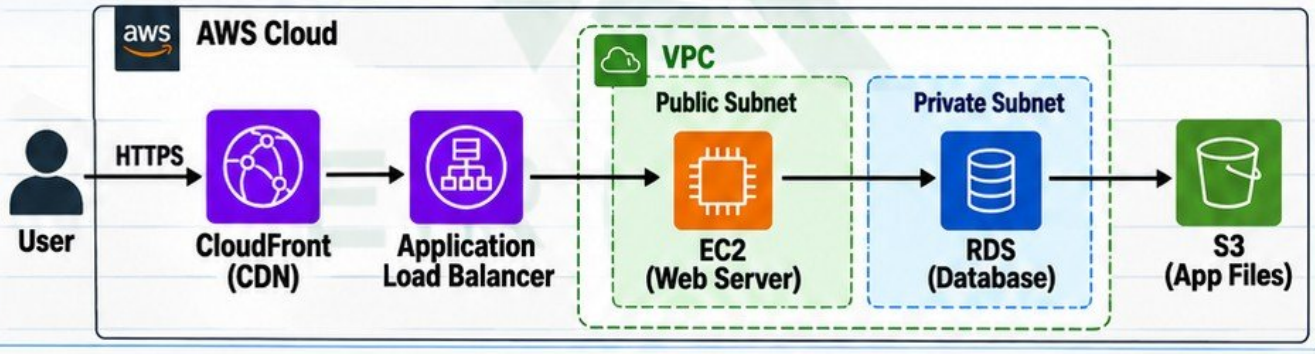
AWS Account Security

- Protect the root user
- Use strong identity and access management (IAM)
- Enable multi-factor authentication (MFA)
- Control who can create or modify resources
- Set up billing alerts and monitoring

Resource Security

- Secure VPC, subnets and security groups
- Protect EC2, S3, databases and other services
- Use encryption and proper access controls
- Monitor and log activity
- Apply security best practices for each service

5. Example AWS Environment



6. Five Assets an Attacker Might Target

1. AWS account (root user, IAM users, roles)
2. S3 data (sensitive files, backups)
3. EC2 instances (application servers)
4. Databases (RDS and credentials)
5. Networking components (VPC, security groups, exposed ports)

7. Key Takeaways

- AWS secures the underlying cloud infrastructure.
- You are responsible for securing your account and resources.
- Security responsibilities vary by service.
- Start with strong identity and access management.
- Always think about what an attacker would target.

02 Lock Down the AWS Root User

VERIQTA

1. What is the AWS Root User?

- The root user is the account owner created when you open an AWS account.
- It has full access to all AWS services and resources.
- It can't be limited by IAM policies.
- It should only be used for account setup, recovery, and critical account-level tasks.



2. Why You Must Not Use It for Daily Work

- ✗ No permission boundaries
- ✗ Can accidentally delete resources
- ✗ High-value target for attackers
- ✗ Actions are difficult to recover from
- ✗ Not suitable for day-to-day engineering work
- ✗ Use IAM users and roles instead

3. Enable Root Multi-Factor Authentication (MFA)

1. Sign in to the AWS root account https://signin.aws.amazon.com
2. Go to Security Credentials
3. Under Multi-factor authentication (MFA) click Assign MFA
4. Choose an MFA device (Authenticator app or hardware device)
5. Follow the on-screen steps to complete setup



4. Verify Recovery Information

- ✓ Verify your root email address
- ✓ Add and verify a mobile phone number
- ✓ Add a recovery email (recommended)
- ✓ Keep this information up to date
- ✓ Use a secure, active email that you control



5. Check for Root Access Keys

1. In the root account, go to Security Credentials
2. Check the Access keys section
3. There should be no active access keys for the root user
4. If keys exist, delete them immediately



6. Use IAM for Everyday Work

- ✓ Create IAM users for human access
- ✓ Use IAM Identity Center (recommender)
- ✓ Use IAM roles for applications and services
- ✓ Grant only the permissions required (least privilege)
- ✓ Reserve the root account for account recovery and critical tasks only

7. Common Failures

- ✗ Using the root account to deploy applications
- ✗ Using the root account for routine administration
- ✗ Keeping root access keys active
- ✗ Not enabling MFA on the root account
- ✗ Using a personal or inactive email for recovery
- ✗ Ignoring security alerts related to root usage

8. Key Takeaways

- ✓ The root user has full access to your AWS account.
- ✓ Enable MFA on the root user.
- ✓ Keep recovery information current.
- ✓ Never use the root account for daily engineering work.
- ✓ Use IAM users, roles, and IAM Identity Center for everyday access.
- ✓ Remove any root access keys.

03 Create Secure Human Access with IAM Identity Center

VERIQTA

1. Key Concepts

Users
People who need access to AWS (e.g., engineers).

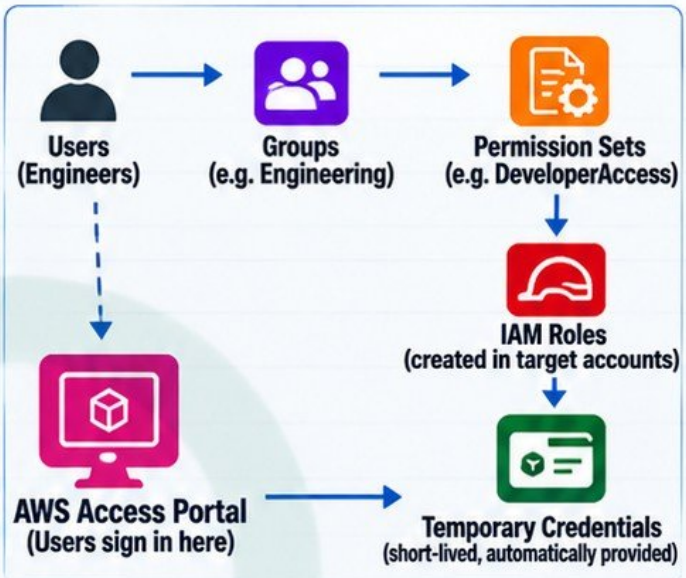
Groups
Collection of users with the same access needs.

Permission Sets
Define what actions users can perform (similar to IAM policies).

Roles
IAM roles created in your AWS accounts based on permission sets.

Temporary Credentials
Short-lived credentials (automatically provided) when users sign in. More secure than permanent access keys.

2. How It Works



3. Set Up IAM Identity Center (AWS Console)

1. Sign in to the AWS Management Console as the account administrator.
2. Open IAM Identity Center.
3. Enable IAM Identity Center (if not already enabled).
4. Choose your identity source (default: AWS Identity Center directory).
5. Create a group (e.g. Engineering).
6. Create or use a permission set (e.g. DeveloperAccess).
7. Assign the group to one or more AWS accounts.

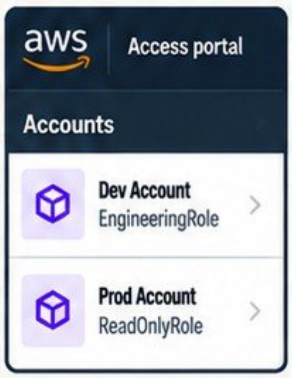
4. Example Permission Set

DeveloperAccess (Example)

- Allows specific developer actions
- Does not provide full administrative access
- Can be customized to follow least privilege
- Creates an IAM role in each assigned account
- Users receive temporary credentials when they sign in

5. Verify Access Through the AWS Access Portal

1. Go to https://start.aws.amazon.com
2. Sign in with your IAM Identity Center user.
3. Choose the AWS account you have access to.
4. Select the role from your assigned permission set.
5. Open the AWS Management Console and confirm which account and role you are using.



6. Common Mistakes

- Creating individual users instead of groups
- Giving overly broad permissions
- Usigg the root user for everyday access
- Not using IAM Identity Center
- Assigning AdministratorAccess unnecessarily

7. Key Takeaways

- Use IAM Identity Center for human access.
- Organize users with groups.
- Assign appropriate permission sets.
- Use least privilege.
- Verify access through the AWS access portal.

04 Understand Least Privilege by Denying Access

VERIQTA

1. How IAM Policies Are Evaluated

1. **Actions**
   What can be done?
   (e.g. s3:GetObject)
2. **Resources**
   Which resource?
   (e.g. specific bucket or *)
3. **Conditions**
   When can it be done?
   (e.g. specific IP, MFA, tags)
4. **Explicit Deny**
   Always takes precedence and overrides any allow.



2. Example Policy Evaluation

Scenario:
A user tries to get a file from an S3 bucket.

IAM Policy Statement

```
{
   "Effect": "Allow",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::my-bucket/*"
}
```

Evaluation:

- Action matches: s3:GetObject
- Resource matches: my-bucket/*
- No explicit deny
- Result: Access Allowed

3. Narrowly Scoped S3 Read-Only Policy (Recommended)

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": ["s3:GetObject",
         "s3:ListBucket"],
         "Resource": [
            "arn:aws:s3:::my-bucket",
            "arn:aws:s3:::my-bucket/*"
         ]
      }
   ]
}
```

What This Does

- Allows listing the bucket
- Allows reading objects
- Only for my-bucket
- No permission to delete, change or upload objects

Use Case

- Applications that only need to read files
- Junior engineers with read-only access

4. Broader Access Policy (Not Recommended)

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": "s3:*",
         "Resource": "*"
      }
] }
```

Risk

- Allows full access to all S3 buckets
- Includes delete, modify and upload
- High risk if the credentials are exposed
- Violates the principle of least privilege

5. Test Policies with IAM Policy Simulator

1. Go to IAM Console → Policy Simulator
2. Select the IAM user or role
3. Choose an action (e.g. s3:GetObject)
4. Enter the resource ARN (e.g. arn:aws:s3:::my-bucket/file.txt)
5. Run the simulation to see Allow or Deny

IAM Policy Simulator (Example)

Action: s3:GetObject

Resource ARN: arn:aws:s3:::my-bucket/file.txt

Simulate

Simulation Result

- Allowed (with the read-only policy)
- Denied (if an explicit deny exists)

6. Common Failure

- Giving AdministratorAccess to an axplication because a permission error appeared.
- Using overly broad permissions for convenience.
- Not using conditions (e.g. IP, MFA, tags).
- Ignoring explicit denies.
- Not testing policies before assigning.

7. Key Takeaways

- Give only the permissions required.
- Use explicit denies to block unwanted access.
- Scope actions and resources carefully.
- Use conditions for extra security.
- Test with IAM Policy Simulator.
- Avoid AdministratorAccess for applications.

05 Stop Using Permanent AWS Access Keys

VERIQTA

1. How Applications Obtain AWS Credentials

1. Access Keys (Long-Lived) Not Recommended

- Access Key ID
- Secret Access Key
- Used by CLI, SDKs and applications
- Long-lived and high risk

2. IAM Roles (Recommended)

- Used by EC2, ECS, Lambda and other services
- Temporary credentials
- Automatically rotated by AWS

3. IAM Identity Center (Recommended)

- For human users
- Sign in through AWS access portal
- Receive temporary credentials
- No permanent access keys

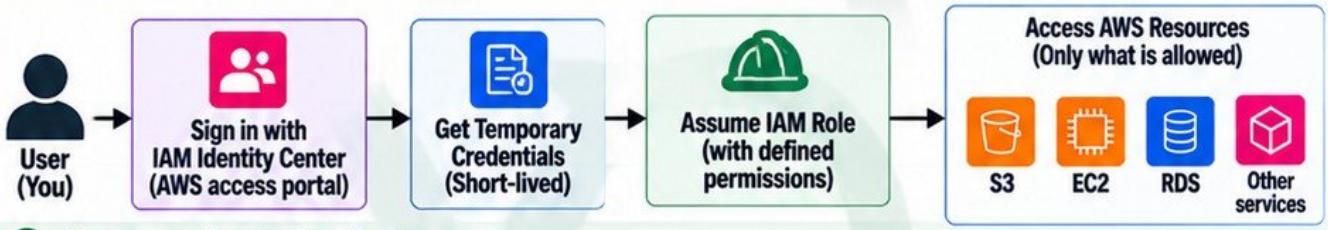
4. Service Credentials

- Used by AWS services
- Temporary and automatic
- Managed by AWS

2. Why Long-Lived Access Keys Are Risky

- Can be exposed in code repositories (GitHub, GitLab, etc.)
- Often stored in scripts and configuration files
- Difficult to rotate and track
- Can be used by attackers from anywhere
- No automatic expiration
- Harder to audit who is using them
- Can lead to full account compromise

3. How Role-Based Access Works



- No permanent access keys for human users
- Temporary credentials with limited permissions
- Access controlled by IAM roles and permission sets

4. Practical: Configure AWS CLI with IAM Identity Center

1. Install and update AWS CLI (v2) (if not already installed)

```
# Check version
aws --version
```

2. Configure AWS CLI with IAM Identity Center

```
aws configure sso
```

3. Follow the prompts in your terminal

```
SSO start URL: https://your-sso-portal.awsapps.com
SSO region: us-east-1
SSO account ID: 123456789012
SSO role name: DevOpsEngineer
CLI profile name: veriqta
```

4. Sign in through the browser (AWS access portal)

```
A browser will open for you to sign in
with IAM Identity Center.
```

5. Verify your identity and permissions

```
aws sts get-caller-identity
```

```
{
  "UserId": "AROAXXXXXXXXXX:ann",
  "Account": "123456789012",
  "Arn": "arn:aws:sts::123456789012:assumed-role/DevOpsEngineer/ann"
}
```

5. Example: No Permanent Keys Needed

Use IAM Identity Center and roles instead of creating access keys.

Recommended (Short-lived credentials)

```
aws sts get-caller-identity
{
    "UserId": "AROAXXXXXXXX:ann",
    "Account": "123456789012",
    "Arn": "arn:aws:sts::123456789012:
      assumed-role/DevOpsEngineer/ann"
}
```

6. Common Failures

- Creating permanent access keys for users
- Storing access keys in GitHub repositories
- Keeping keys in scripts or configuration files
- Giving overly broad permissions
- Not rotating or deleting unused keys

7. Key Takeaways

- Use IAM Identity Center for human access.
- Use IAM roles for applications and services.
- Avoid permanent access keys.
- Follow least privilege.
- Keep credentials out of code and configuration.

06 Design a VPC That Limits Exposure

VERIQTA

1. What is a VPC?

- A Virtual Private Cloud (VPC) is your isolated network in AWS.
- You control IP address ranges, subnets, route tables and network security.
- It provides a logically isolated environment for your resources.
- A VPC does not automatically make your resources secure.

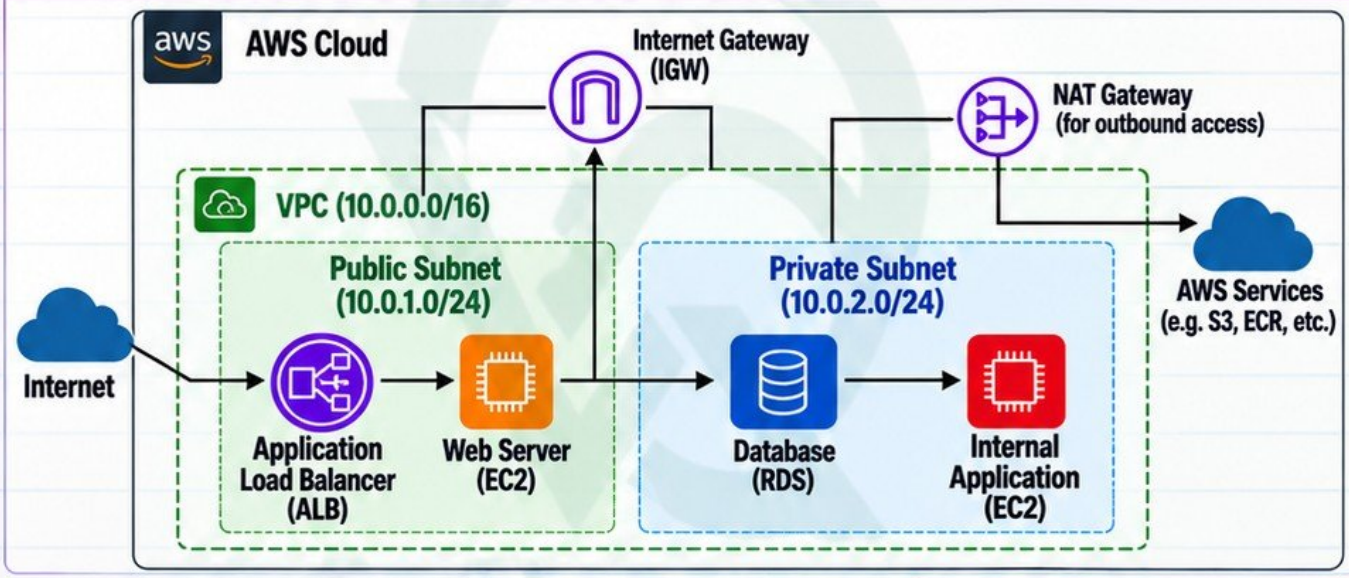


Amazon VPC
Your private network in AWS

2. Key Components

Subnets	Divide your VPC into public and private subnets.
Route Tables	Control where network traffic goes.
Internet Gateway (IGW)	Connects your VPC to the internet (for public subnets).
NAT Gateway	Allows private subnet resources to access the internet (outbound only).
Security Groups	Act as a firewall for your resources.

3. Example: Two-Subnet Application Environment



4. Route Table Example (Public Subnet)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Internet Gateway (IGW)

5. Route Table Example (Private Subnet)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	NAT Gateway

6. Which Resources Need Internet Access?

- ✓ Application Load Balancer (public)
- ✓ Web server (if serving internet traffic)
- ✓ Private resources (via NAT, for updates)
- ✗ Database (should not be publicly accessible)
- ✗ Internal services (keep in private subnet)

7. Common Failures

- ✗ Assuming resources in a VPC are automatically secure
- ✗ Placing databases in public subnets
- ✗ Allowing 0.0.0.0/0 access in security groups
- ✗ Not using a NAT gateway for private subnet outbound access
- ✗ Misconfigured route tables leading to unintended internet access

8. Key Takeaways

- ✓ Use public subnets only for resources that need inbound internet access.
- ✓ Keep databases and backend services in private subnets.
- ✓ Use route tables to control traffic flow.
- ✓ Use a NAT gateway for private subnet outbound access.
- ✓ A VPC provides isolation, but you must still configure security settings correctly.

07 Build Security Groups That Enforce Boundaries

VERIQTA

1. What are Security Groups?

- Virtual firewall for your resources (EC2, RDS, ALB, etc.).
- Controls inbound and outbound traffic.
- Stateful: If you allow incoming traffic, return traffic is automatically allowed.
- Rules are based on protocol, port and source/destination.

2. Inbound vs Outbound Rules

Inbound Rules

- Control traffic coming into your resource.
- You must explicitly allow it.

Outbound Rules

- Control traffic leaving your resource.
- Allow only what is required.

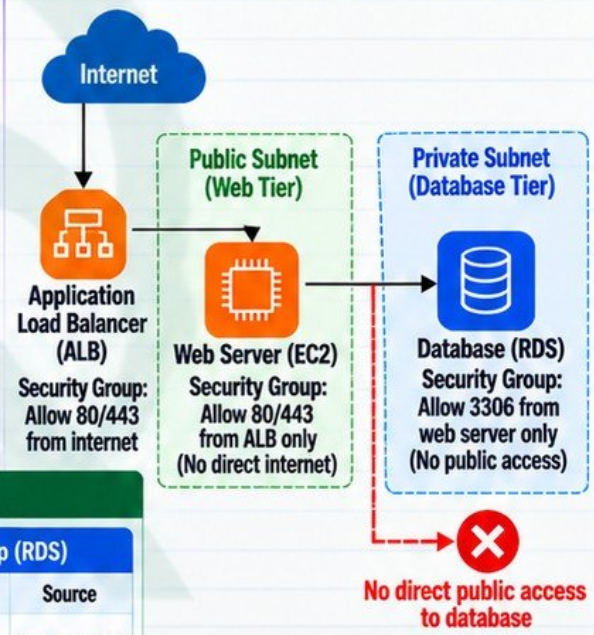
3. Stateful Filtering

- Security groups are stateful.
- When you allow inbound traffic (e.g. HTTPS), the response traffic is automatically allowed.
- You do not need to create separate outbound rules for return traffic.

4. Security Groups vs Network ACLs

Feature	Security Groups	Network ACLs
Level	Instance level	Subnet level
Stateful	Yes (automatic return traffic)	No (stateless)
Rules	Allow rules only	Allow and Deny rules
Rule order	Not evaluated in order	Evaluated in numbered order
Typical use	Control traffic to resources (e.g. EC2, RDS, ALB)	Additional subnet-level protection
Best practice	Use security groups for resource access control	Use NACLs for extra layer or compliance requirements

5. Two-Tier Application Architecture



6. Example Security Group Rules

Web Server Security Group (EC2)

Type	Protocol	Port	Source
HTTP	TCP	80	ALB SG
HTTPS	TCP	443	ALB SG
SSH	TCP	22	(your IP) (optional)
All Outbound	All	All	0.0.0.0/0 (or as required)

Database Security Group (RDS)

Type	Protocol	Port	Source
MySQL (or as needed)	TCP	3306	Web Server SG
All Outbound	All	All	0.0.0.0/0 (or as required)

7. Troubleshooting: Application Cannot Connect to Database

1. Check the database security group allows traffic from the web server security group.
2. Verify the correct port (e.g. 3306 for MySQL).
3. Confirm the database is in a private subnet (no public access).
4. Check the web server can reach the database (use telnet or nc).
5. Ensure the database is running and accepting connections.
6. Review VPC route tables, NACLs and DNS settings.

Test Connectivity from Web Server

```
# Test database port
nc -zv <db-endpoint> 3306

# or
telnet <db-endpoint> 3306

# Expected (if open):
Connection to <db-endpoint>
port 3306 [tcp/mysql] succeeded!
```

8. Common Failures

- ✗ Allowing 0.0.0.0/0 (public access) to the database.
- ✗ Using the same security group for all resources.
- ✗ Overly broad inbound or outbound rules.
- ✗ Relying on a VPC for security without configuring security groups.
- ✗ Blocking outbound traffic needed for updates or package downloads.

9. Key Takeaways

- Use security groups to enforce precise access boundaries.
- Allow only the traffic required.
- Keep databases in private subnets.
- Use security group references instead of IP addresses where possible.
- A VPC provides isolation, but you must still configure security groups correctly.

08 Secure EC2 Without Opening SSH to the Internet

VERIQTA

1. Key Concepts

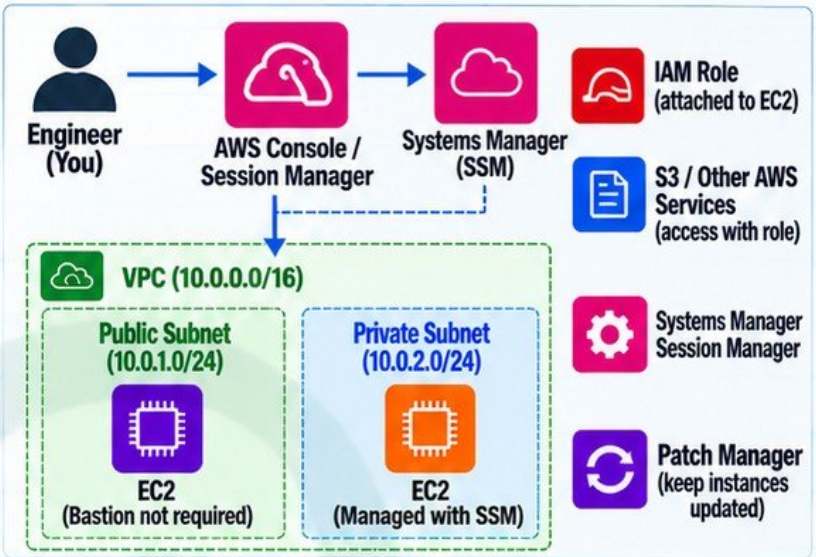
EC2 Instance Roles
IAM roles attached to EC2 instances to give temporary credentials to AWS services (no access keys).

Metadata Credentials
EC2 instances can get temporary credentials from the instance metadata service (IMDS).

Patching
Keep your EC2 instances and OS packages updated to fix security vulnerabilities.

AWS Systems Manager Session Manager
Allows secure shell access to EC2 instances without opening port 22 to the internet.

2. Architecture: Secure EC2 Access



3. EC2 Instance Role vs Access Keys

Feature	EC2 Instance Role (Recommended)	Access Keys (Not Recommended)
Credentials	✓ Temporary, automatically provided by AWS	✗ Long-lived keys (manual)
Security	✓ No keys to manage	✗ Can be leaked or exposed
Rotation	✓ Automatic	✗ Manual
Use case	✓ EC2 accessing AWS services (S3, DynamoDB, etc.)	✗ Not required for EC2
Best practice	✓ Use IAM roles	✗ Avoid storing keys on instances

4. How to Connect with Session Manager

1. Ensure the instance has the SSM agent installed (most Amazon Linux/Ubuntu AMIs already include it).
2. Attach an IAM role with the required Systems Manager permissions to the EC2 instance.
3. Ensure the instance has network access to the internet (through a NAT gateway or internet gateway).
4. In the **AWS console**, open Systems Manager → **Session Manager**.
5. Click **Start session** and connect to your instance.

5. Prerequisites (IAM Role Permissions)

Minimum IAM Policy for EC2 Role (to use Session Manager)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ssm:UpdateInstanceInformation",
        "ssmmessages:CreateControlChannel",
        "ssmmessages:CreateDataChannel",
        "ssmmessages:OpenControlChannel",
        "ssmmessages:OpenDataChannel"
      ], "Resource": "*"
    }
}
```

6. Test Connection

```
# Start a session from AWS CLI
aws ssm start-session \
   --target i-0123456789abcdef0
```

```
# Check you are on the instance
whoami
ec2-user
```

```
# Check instance metadata access
curl http://169.254.169.254/
latest/meta-data/iam/security-credentials/
<role-name>
```

7. Common Failures

- ✗ Opening port 22 (SSH) to the internet.
- ✗ Instance role is missing or has insufficient permissions.
- ✗ SSM agent is not installed or not running.
- ✗ Instance has no internet access (no NAT gateway).
- ✗ Blocking SSM endpoints with security groups or network ACLs.
- ✗ Assuming an instance is secure just because it is inside a VPC.

8. Key Takeaways

- ✓ Use EC2 instance roles instead of access keys.
- ✓ Use AWS Systems Manager Session Manager for secure shell access (no port 22).
- ✓ Keep your instances and packages updated with Patch Manager.
- ✓ Ensure the instance can reach SSM endpoints.
- ✓ Do not store access keys in GitHub repositories, scripts, or configuration files.
- ✓ A VPC provides isolation, but you must still configure the correct security settings.

09 Prevent Accidental S3 Data Exposure

VERIQTA

1. Key Concepts

S3 Buckets
Containers for storing objects such as files, logs, backups, etc.

Bucket Policies
Define who can access the bucket and what actions are allowed.

Public Access Controls
Block public access to prevent unintended exposure.

Object Ownership
Control who owns uploaded objects and how permissions are applied.

Encryption
Protect data at rest using SSE-S3 or SSE-KMS.

2. S3 Access Types (Better vs Risky)

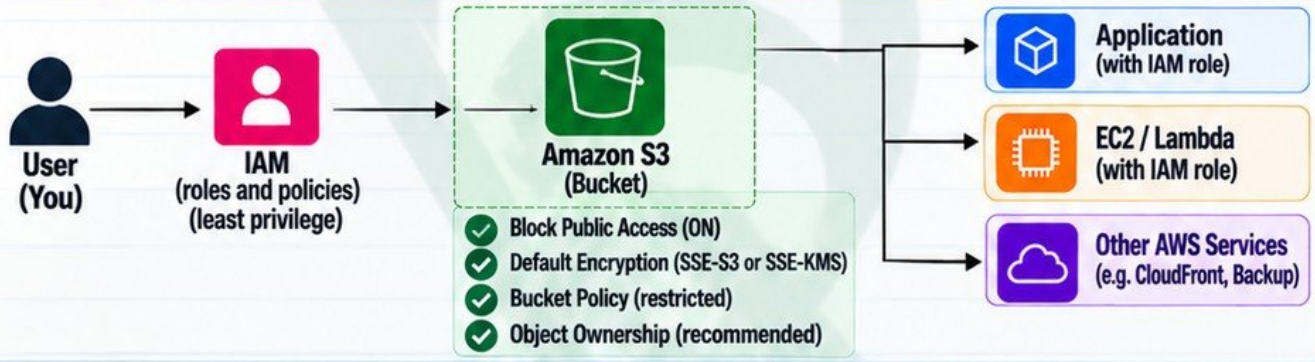
Recommended (Private)

- Block public access enabled
- Access via IAM roles and policies
- Encrypted at rest (SSE-S3 or SSE-KMS)
- Use bucket policies with least privilege
- Use presigned URLs for temporary file access (if needed)

Risky (Public Exposure)

- Bucket set to public
- Public read or write access
- Disabling Block Public Access
- Using * in bucket policies
- Storing sensitive data without encryption
- Sharing the entire bucket for one file

3. Example: Secure S3 Bucket Architecture



4. Practical: Configure a Secure S3 Bucket

1. Create an S3 bucket (with a unique name)
2. Enable Block Public Access (all settings)
3. Enable default encryption (SSE-S3 or SSE-KMS)
4. Set object ownership (Recommended: Bucket owner enforced)
5. Add a bucket policy (to allow only specific access)
6. Upload a test file and verify the settings

5. Example Bucket Policy (Allow specific access)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS":
        "arn:aws:iam::123456789012:role/MyAppRole"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

6. Verify the Settings

- Block Public Access = Enabled (all four settings)
- Default Encryption = Enabled
- Object Ownership = Bucket owner enforced (or appropriate setting)
- Bucket Policy = Only intended access
- No public access (test with AWS CLI)
- File can be accessed only by authorized users or roles

7. Common Failures

- Making the entire bucket public to share one file.
- Disabling Block Public Access.
- Using overly broad bucket policies (*).
- Storing sensitive data without encryption.
- Giving public read access to logs, backups or application files.
- Not reviewing permissions after testing.

8. Key Takeaways

- Keep S3 buckets private by default.
- Use IAM roles and bucket policies with least privilege.
- Enable Block Public Access on every bucket.
- Use encryption for all data.
- Use presigned URLs or CloudFront to share files instead of making buckets public.
- Regularly review bucket policies and access permissions.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

10 Protect Secrets and Understand KMS Encryption

VERIQTA

1. Key Concepts

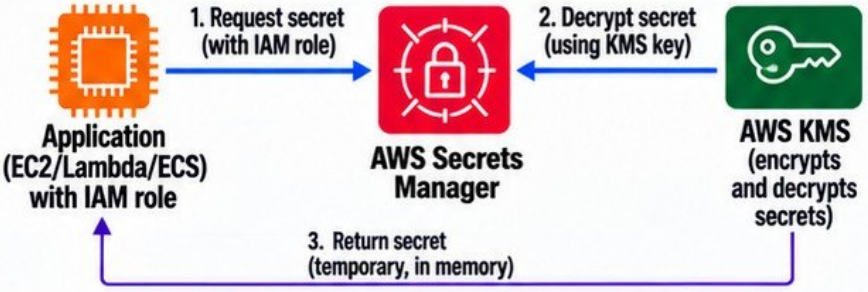
AWS KMS
Managed service for creating and controlling encryption keys used to protect your data.

AWS Secrets Manager
Stores and manages sensitive information such as database passwords, API keys and rotates them automatically.

Encryption at Rest
Data is encrypted when stored (e.g. S3, EBS, RDS) using KMS keys.

Secret Rotation
Automatically updates secrets on a schedule to reduce risk from long-lived credentials.

2. How It Works Together



Example Use Cases

- Database passwords
- API keys
- Third-party credentials
- Application configuration
- Email credentials
- Other sensitive data

3. Hardcoded Password vs Secrets Manager

Hardcoded Password (Risky)

```
# BAD PRACTICE
import os

DB_PASSWORD = "MyP@ssw0rd123"

# Used directly in the application
connect_to_database(DB_PASSWORD)
```

- Password stored in code
- Can be exposed in GitHub repositories
- Difficult to rotate
- Same password used for long periods
- High risk of unauthorized access

Using AWS Secrets Manager (Recommented)

```
import boto3
import json

client = boto3.client('secretsmanager')

response = client.get_secret_value(
    SecretId='prod/db-credentials'
)

secret = json.loads(response['SecretString'])
DB_PASSWORD = secret['password']
connect_to_database(DB_PASSWORD)
```

- Password stored securely in Secrets Manager
- Retrieved at runtime using IAM role
- No hardcoded credentials in code
- Can be rotated automatically
- Accese controlled by IAM policies

4. KMS and Secrets Manager Details

AWS KMS
- Creates and manages encryption keys (customer managed or AWS managed).
- Used for encryption at rest across AWS services.
- Controls who can encrypt and decrypt data using IAM policies.
- Supports key rotation.
- Provides audit logs in AWS CloudTrail.

AWS Secrets Manager
- Stores secrets (passwords, API keys, tokens, etc.).
- Encrypts secrets using AWS KMS.
- Allows automatic secret rotation.
- Controls access with IAM policies.
- Integrates with AWS services and applications.
- Provides audit logs and version history.

5. Practical: Create and Use a Secret

1. Create a KMS key (or use AWS managed key).
2. Create a secret in AWS Secrets Manager (e.g. database credentials).
3. Attach an IAM role with the required permissions (secretsmanager:GetSecretValue).
4. Retrieve the secret using AWS CLI or SDK.
5. Verify that the application can connect using the secret.

6. Example IAM Permission (Least Privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:
    us-east-1:123456789012:secret:prod/db-*"
    }
  ]
}
```

7. Common Failures

- Storing secrets in code or GitHub repositories.
- Giving overly broad permissions (e.g. secretsmanager:*).
- Not enabling rotation for long-lived credentials.
- Using the same secret across multiple environments.
- Forgetting that KMS policies also control decryption access.

8. Key Takeaways

- Use AWS Secrets Manager instead of hardcoded secrets.
- Encrypt data at rest using AWS KMS.
- Apply least privilege access policies.
- Enable secret rotation.
- Remember: encryption protects data, but IAM policies determine who can decrypt or retrieve it.

11 Use CloudTrail to Find Out Who Changed What

VERIQTA

1. Key Concepts

AWS CloudTrail
Records API activity in your AWS account.

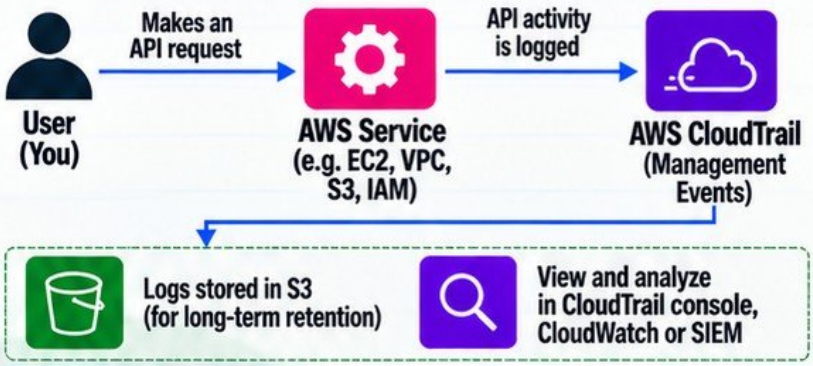
API Activity
Logs management events, data events (optional) and insights for security monitoring.

Management Events
Record control plane actions (e.g. creating, modifying or deleting AWS resources).

Event History
Shows recent events in the AWS console (last 90 days).

Audit Trails
Essential for security, compliance, incident investigation and troubleshooting.

2. How CloudTrail Works



Types of CloudTrail Events

Management Events
- Control plane operations (e.g. Create, Modify, Delete)
- Enabled by default
- Used for auditing and compliance

Data Events
- Record resource level activity (e.g. S3 object access, Lambda invocations)
- Disabled by default
- Useful for detailed auditing of data access

Insights Events
- Detect unusual API activity
- Helps identify potential security issues
- Enabled separately

3. Practical: Run an AWS CLI Command

1. Run a harmless command to describe your security groups.

```
# AWS CLI command
aws ec2 describe-security-groups \
    --region us-east-1
```

2. Note the time you ran the command (e.g. 2026-10-30 12:45).

4. Find the Event in CloudTrail

1. Open the AWS Console and go to CloudTrail → Event history.
2. Filter by Event name: DescribeSecurityGroups
3. Check the event details:

- Event time (matches when you ran the command)
- User identity (your IAM user or role)
- Event name (DescribeSecurityGroups)
- Source IP address
- Request parameters
- Response elements
- AWS service (ec2.amazonaws.com)

5. Example CloudTrail Event


```
{
  "eventTime": "2026-10-30T12:45:32Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "DescribeSecurityGroups",
  "awsRegion": "us-east-1",
  "sourceIPaddress": "203.0.113.25",
  "userIdentity": {
     "type": "AssumedRole",
     "arn": "arn:aws:sts::123456789012:
     assumed-role/EngineerRole/ann"
  },
  "requestParameters": { },
  "responseElements": {},
  "eventType": "AwsApiCall",
  "managementEvent": true
}
```


6. Console View (Event History)

CloudTrail > Event history

Event name | DescribeSecurityGroups

Event time	Event name	AWS service	User identity type
2026-10-30 12:45:32	DescribeSecurityGroups	EC2	AssumedRole
2026-10-30 11:12:17	RunInstances	EC2	AssumedRole
2026-10-30 10:03:54	CreateSecurityGroup	EC2	IAMUser

7. Why Audit Logging Matters

- Helps you know who changed what and when.
- Detects unauthorized or suspicious activity.
- Supports incident investigation and root cause analysis.
- Required for compliance (e.g. ISO 27001, SOC 2).
- Provides accountability across teams and environments.

8. Common Failures

- Relying only on Event history (last 90 days).
- Not enabling a multi-region CloudTrail.
- Not storing logs in a secure S3 bucket with proper access controls.
- Disabling or deleting CloudTrail logs.
- Not reviewing logs regularly.

9. Key Takeaways

- Use CloudTrail to track and audit all API activity.
- Management events are enabled by default, but configure durable logging to S3.
- Event history is useful but not a replacement for long-term, protected audit logs.
- Regularly review CloudTrail logs and set up alerts for suspicious activity.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

12 Detect Suspicious Activity with GuardDuty

VERIQTA

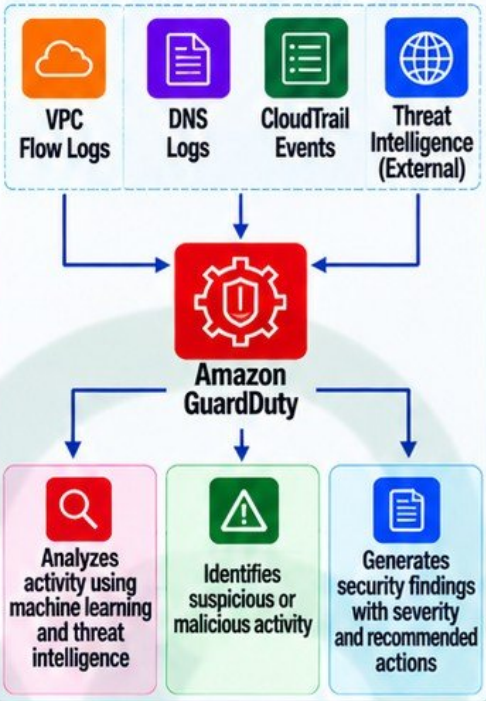
1. What is Amazon GuardDuty?

Amazon GuardDuty
A threat detection service that continuously monitors your AWS environment for suspicious and malicious activity using machine learning, anomaly detection and threat intelligence.

Analyzes Multiple Sources
Monitors VPC Flow Logs, DNS Logs, CloudTrail events and other supported sources to identify potential threats.

Helps You Detect
Unauthorized access, compromised instances, unusual API activity, malicious IP addresses and other suspicious behaviour.

2. How GuardDuty Works



3. Supported Activity Sources

VPC Flow Logs
Detects unusual network traffic (e.g. communication with malicious IP addresses).

DNS Logs
Identifies suspicious domain requests (e.g. known malicious domains).

CloudTrail Events
Detects unusual API activity and unauthorized changes.

EKS Audit Logs
Monitors activity in Amazon EKS clusters.

S3 Data Events (Optional)
Monitors unusual access to sensitive data in S3.

Threat Intelligence Feeds
Uses known malicious IP addresses, domains and threat actor information.

4. Generate an Official Sample Finding

1. Open the Amazon GuardDuty console.
2. Go to Findings → Sample findings.
3. Select a sample finding to view the details (e.g. UnauthorizedAccess:IAMUser/ConsoleLogin).
4. Review the finding information, including severity, affected resource and recommended response.

5. Example GuardDuty Finding

```
{
  "title": "UnauthorizedAccess:IAMUser/ConsoleLogin",
  "description": "Instance i-0123456789abcdef0
    was involved in an unauthorized API call",
  "severity": 7,
  "service": { "serviceName": "guardduty" },
  "resource": {
    "instanceDetails": {
      "instanceId": "i-0123456789abcdef0",
      "instanceType": "t3.micro"
    }
  },
  "recommendation": "Review the user activity,
    verify if the login was expected and take
    appropriate action."
}
```

6. Understanding the Finding

Field	What It Tells You
Title	Type of suspicious activity detected.
Description	Detailed explanation of the finding.
Severity	Level of risk (1 = Low, 10 = High).
Affected resource	The AWS resource involved (e.g. EC2 instance, IAM user, IP address).
Recommendation	Suggested steps to investigate and respond.

7. Severity Levels

1 – 3	Low	Unusual activity with low risk.
4 – 6	Medium	Potentially suspicious activity.
7 – 8	High	Likely malicious or unauthorized activity.
9 – 10	Critical	Highly likely malicious activity that requires immediate attention.

8. Recommended Response Steps

1. Review the finding details and affected resource.
2. Investigate the activity (check CloudTrail logs, VPC Flow Logs, etc.).
3. Determine if the activity is expected or authorized.
4. Take appropriate action (e.g. isolate instance, revoke credentials, block malicious IP).
5. Mark the finding as resolved or create a Security Hub issue for tracking.

9. Beginner Mistakes

- Treating every GuardDuty finding as proof of an actual attack.
- Ignoring low or medium severity findings that may indicate early signs of risk.
- Not investigating the affected resource.
- Disabling GuardDuty because of false positives.
- Relying only on GuardDuty and not reviewing CloudTrail logs.

10. Key Takeaways

- GuardDuty continuously monitors multiple data sources.
- It uses machine learning and threat intelligence to detect suspicious activity.
- Findings provide valuable information, but you must investigate them.
- Severity helps you prioritize your response.
- Not all findings are real attacks; investigate before taking action.
- Use GuardDuty together with CloudTrail and other security tools for better visibility and protection.

13 Catch Configuration Drift with AWS Config

VERIQTA

1. Key Concepts

AWS Config
A continuous monitoring service that records and evaluates the configuration changes in your AWS account.

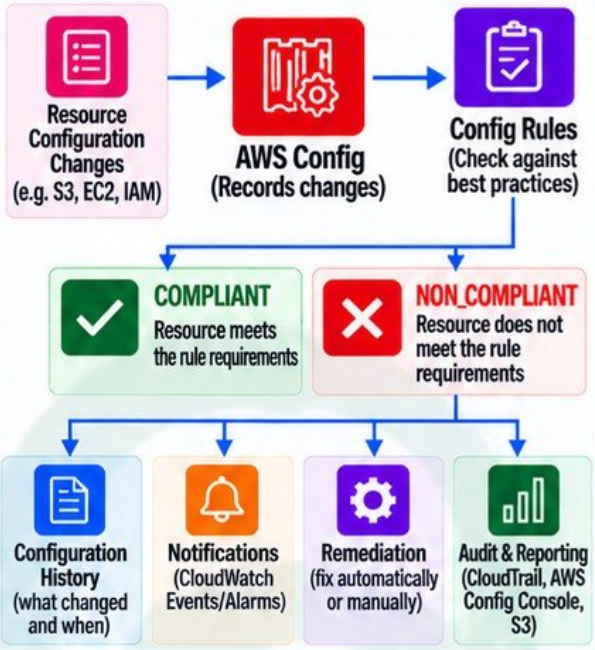
Configuration Drift
Security settings can change over time, making a resource that was secure yesterday exposed today.

Config Rules
Evaluate resources against your desired configuration (e.g. S3 public access disabled).

Compliance Status
Shows whether a resource is COMPLIANT or NON_COMPLIANT with a rule.

Remediation
Automatically or manually fix noncompliant resources.

2. How AWS Config Works



3. Benefits of AWS Config

Continuous Monitoring
Tracks configuration changes across your AWS resources.

Detects Unintended Changes
Identifies when resources drift from your desired security configuration.

Supports Compliance
Helps meet security and regulatory requirements (e.g. ISO 27001, SOC 2).

Works Across Accounts and Regions
Monitors multiple accounts and regions from a central location.

Provides Historical View
Shows who changed what and when.

4. Practical: Check S3 Public Access Config Rule

1. Open the AWS Config console.
2. Go to Rules → AWS managed rules.
3. Search for s3-bucket-public-read-prohibited (or similar S3 public access rule).
4. Select the rule and review the compliance status.
5. Click a noncompliant resource to see the details and recommended remediation steps.

Example Rule in AWS Console

s3-bucket-public-read-prohibited
Checks whether your S3 buckets prohibit public read access.

Rule status	Compliance type	Scope
ACTIVE	PERIODIC	All resources

Compliance status

5	1	0
Compliant	Non-compliant	Not applicable

Resource ID	Resource type	Compliance status
my-private-bucket	S3 Bucket	COMPLIANT
my-public-bucket	S3 Bucket	NON_COMPLIANT
backup-bucket	S3 Bucket	COMPLIANT

5. Example: Noncompliant Resource Details

Resource details	NON_COMPLIANT
Resource ID	my-public-bucket
Resource type	AWS::S3::Bucket
Compliance reason	The bucket allows public read access (ACL or bucket policy).
Annotation	Public access is not blocked.

Recommended remediation

- Enable Block Public Access for the bucket.
- Remove public read permissions from the bucket policy and ACL.
- Follow AWS best practices for S3 security.

6. Example: Rule Evaluation (Simplified)



7. Troubleshooting: Drift vs Exploitation

Configuration Drift
(Noncompliant but not an attack)

- Occurs when someone changes a setting (intentionally or by mistake).
- Resource is misconfigured (e.g. S3 bucket becomes public).
- Detected by AWS Config rules.
- Does not automatically mean an attacker has accessed the resource.

Actual Exploitation
(Confirmed or suspected attack)

- Evidence of unauthorized access in logs (e.g. CloudTrail, GuardDuty).
- Unusual or malicious activity detected.
- May require further investigation using CloudTrail, GuardDuty, and other logs.
- Configuration drift can be a risk factor, but it is not proof of exploitation.

8. Key Takeaways

- AWS Config continuously monitors your resource configurations.
- A secure resource can become exposed if settings change.
- Use AWS managed rules (e.g. S3 public access rules).
- Investigate noncompliant resources, but do not assume an attack.
- Combine AWS Config with CloudTrail and GuardDuty for better visibility.
- Set up notifications and remediation to reduce risk.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

14 Build Your First Security Findings Dashboard

VERIQTA

1. Key Concepts

AWS Security Hub
Provides a centralized view of security findings and compliance status across your AWS environment.

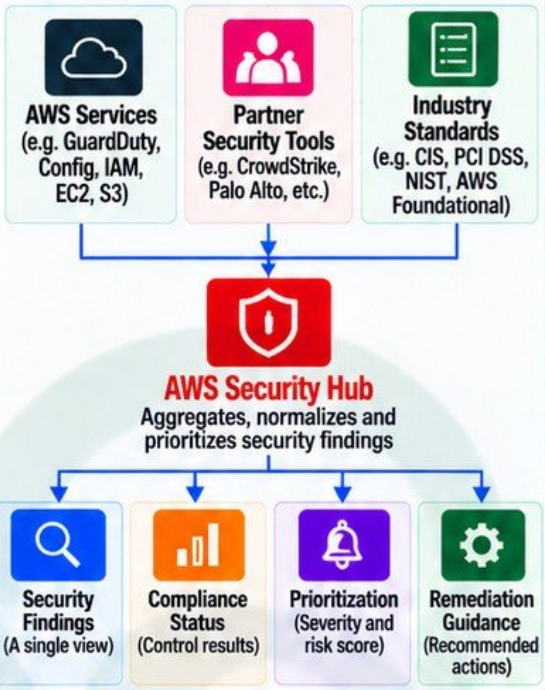
Cloud Security Posture Management (CSPM)
Continuously checks your environment against security standards and best practices.

Security Controls
Predefined rules that evaluate your resources for security risks and compliance.

Findings
Security issues or violations discovered by Security Hub (from AWS services and partner integrations).

Remediation Priorities
Helps you focus on the most important risks first based on severity and impact.

2. How Security Hub Works



3. Security Hub Insights

Centralized Visibility
View findings from multiple AWS services and partners in one dashboard.

Compliance Monitoring
Check your compliance status against industry standards (CIS, PCI DSS, NIST, etc.).

Risk Prioritization
Focus on the most critical findings based on severity, impact and exploitability.

Actionable Guidance
Provides recommended remediation steps for each finding.

Multi-Account Support
Monitor and manage findings across multiple AWS accounts and regions.

Continuous Monitoring
Automatically ingests new findings as your environment changes.

4. Practical: Review Security Hub Findings

1. Open the AWS Security Hub console.
2. Go to Findings to view detected security issues.
3. Review the severity, title and description of each finding.
4. Identify the affected resource (e.g. S3 bucket, EC2 instance).
5. Check the recommended remediation steps.
6. Decide which findings require urgent attention.

5. Example Security Hub Dashboard

Security Hub overview — Last 30 days

12 Critical	28 High	45 Medium	16 Low	101 Informational

Findings by resource type

S3	32
EC2	18
IAM	14
RDS	10
VPC	8
Others	5



Critical	12
High	28
Medium	45
Low	16
Informational	101

6. Example Failed Controls

Security control	Status	Severity	Affected resource
S3 buckets should have block public access enabled	FAILED	High	my-app-backup (S3 bucket)
EC2 instances should not have a public IP	FAILED	Critical	i-0abc123def456 (EC2 instance)
IAM users should not have console access keys	FAILED	High	dev-user-1 (IAM user)
RDS instances should have encryption enabled	FAILED	Medium	database-1 (RDS instance)

7. Determine Urgent Findings

Critical (Act immediately)	• Publicly accessible resources • Exposed sensitive data • High risk of exploitation
High (Fix soon)	• Important security controls not enabled • Misconfigurations that increase risk
Medium (Plan to fix)	• Lower risk issues • Improve security posture
Low (Review)	• Informational or best practice recommendations • Lower impact

8. Production Lessons

- A dashboard shows findings, but it does not always mean your account has been hacked.
- Investigate each finding to understand the actual risk.
- Prioritize based on severity, context and business impact.
- Some findings are misconfigurations, not active exploitation.
- Use the recommended remediation steps and verify the fix.
- Continuously monitor and review new findings.

9. Key Takeaways

- AWS Security Hub provides a centralized view of security findings.
- It helps you understand your security posture and compliance status.
- Not all findings mean an attacker has compromised your account.
- Prioritize and investigate findings based on severity and impact.
- Use recommended remediation steps and verify the risks.
- Integrate Security Hub with other AWS services for continuous monitoring.

15 Investigate an Unexpected AWS Security Change

VERIQTA

1. Scenario Overview

A security group that was previously restricted suddenly allows unrestricted inbound SSH (port 22) from the internet (0.0.0.0/0).

Your Goal

Find out when the change happened, who made the change, how it happened, and fix it safely.

Key Skills

Inspect security groups, use CloudTrail, identify the principal responsible, remediate and verify.

2. High Level Flow

1. Detect the unexpected change (security group allows 0.0.0.0/0) on port 22)
2. Inspect the security group (current rules)
3. Find the CloudTrail event (who, when, what changed)
4. Identify the principal responsible (IAM user/role)
5. Remediate and verify (remove rule and check other resources)

3. What an Unrestricted SSH Rule Looks Like

Inbound rules — Edit inbound rules

Type	Protocol	Port range	Source	Description
SSH	TCP	22	0.0.0.0/0	-

This allows SSH access from anywhere on the internet and is a security risk.

4. Inspect the Security Group

1. Open the EC2 console and go to Security Groups.
2. Find the affected security group.
3. Check the Inbound rules for any 0.0.0.0/0 entry on port 22.
4. Review the description and compare with expected configuration.

AWS CLI (example)

```
aws ec2 describe-security-groups \
 --group-ids sg-0123456789abcdef0 \
--query "SecurityGroups[0].IpPermissions"
--output table
```

5. Find the CloudTrail Event

1. Open the CloudTrail console and go to Event history.
2. Filter by Event name: AuthorizeSecurityGroupIngress
3. Set the time range around when the change occurred.
4. Review the event details to see the principal, source IP, and the exact rule that was added.

CloudTrail Event (example)

```
{
  "eventTime": "2026-10-30T12:15:32Z",
  "eventName": "AuthorizeSecurityGroupIngress",
  "userIdentity": {
    "type": "IAMUser",
    "userName": "dev-user"
  },
  "requestParameters": {
    "groupId": "sg-0123456789abcdef0",
    "ipPermissions": [
      {
        "ipProtocol": "tcp",
        "fromPort": 22,
        "toPort": 22,
        "ipRanges": [{"cidrIp": "0.0.0.0/0"}]
      }
    ]
}
```

6. Identify the Principal

Look at the userIdentity section in the CloudTrail event to find:

- IAM user name
- IAM role name
- Assumed role (federated user)
- Root user (should be avoided)

Check the sourceIPAddress to see where the request came from.

Review additional details such as user agent and MFA status.

Determine if the change was intentional or unauthorized.

7. Remediate the Issue

1. Remove the unnecessary 0.0.0.0/0 SSH rule from the security group.
2. Replace with a restricted source (e.g, your office IP or a bastion host).
3. Follow your organization's change management process.

AWS CLI (remove the rule)

```
aws ec2 revoke-security-group-ingress \
 --group-id sg-0123456789abcdef0 \
--protocol tcp --port 22 --cidr 0.0.0.0/0
```

8. Verify the Fix

1. Check the security group again and confirm the 0.0.0.0/0 rule is removed.
2. Test that SSH is no longer open from the internet.
3. Review recent CloudTrail events to ensure no similar changes were made to other security groups.
4. If using Security Hub or AWS Config, check for related findings or noncompliant resources.

9. Check for Wider Impact

Look for similar changes in other security groups, accounts or regions.

Check if other resources (EC2, RDS, load balancers) were affected.

Use CloudTrail, Security Hub and AWS Config to search for related changes in the same time period.

10. Key Takeaways

- Security settings can change at any time.
- CloudTrail shows who made the change and when.
- Always verify and investigate unexpected changes.
- Not every change is malicious, but all unexpected changes must be reviewed.
- After remediation, check for similar changes across other resources.

16 Create an Alert for Dangerous AWS Activity

VERIQTA

1. Key Concepts

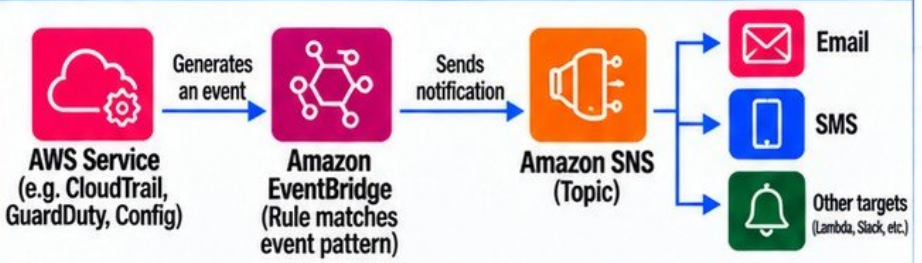
Amazon EventBridge
Monitors events from AWS services and routes them to targets based on rules.

Amazon SNS
Sends notifications through email, SMS or other protocols when an event occurs.

Security Event Routing
Filters specific AWS events (e.g. unauthorized API calls, security group changes) and routes them to an SNS topic or other target.

Actionable Alerts
Helps you detect and respond to dangerous activity quickly.

2. How It Works



3. Example: Event Pattern for a Security Event

```
{
  "source": ["aws.ec2"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventName": ["AuthorizeSecurityGroupIngress"],
    "errorCode": ["null"],
    "sourceIPAddress": [{ "notEquals": "AWS Internal" }]
  }
}
```

This matches events where:

- The EC2 service is involved
- The API call is AuthorizeSecurityGroupIngress
- The event was successful (no error)
- The source IP is not an AWS internal address

4. Practical: Create a Test Alert

1. Create an SNS topic for notifications.
2. Subscribe your email (or phone) to the topic and confirm the subscription.
3. Create an EventBridge rule with an event pattern for a security event.
4. Set the target to your SNS topic.
5. Generate a test event (e.g. make a harmless API call or use a custom event).
6. Verify that you receive the notification.

5. Test with a Custom Event (CLI)

```
# Put a custom test event
aws events put-events --entries '[
  {
    "Source": "custom.security",
    "DetailType": "Test Security Event",
    "Detail": "{\"message\":\"This is
  a test event from EventBridge\"}"
  }
]'
```

Expected result:
You should receive an email (or SMS) notification from your SNS topic.

6. Example SNS Email Notification

AWS Notification Message

From: no-reply@sns.amazonaws.com

Subject: Test Security Event

Message:

```
{
  "message": "This is a test event
 from EventBridge",
  "detail-type": "Test Security Event",
  "source": "custom.security",
  "time": "2026-10-30T12:00:00Z"
}
```

7. Common Security Events to Monitor

Event	Service	Why It Matters
Unauthorized API calls	CloudTrail	May indicate compromised credentials.
Security group changes	EC2	Can expose resources to the internet.
Root user activity	IAM	High risk and should be rare.
S3 bucket policy changes	S3	May lead to data exposure.
GuardDuty findings	GuardDuty	Detects potential threats.
Config rule violations	AWS Config	Indicates noncompliant resources.

8. Common Failures

- Creating alerts without testing delivery.
- Using an incorrect event pattern that does not match real events.
- Not setting up a clear response process.
- Sending alerts to an unmonitored email address.
- Creating too many alerts (alert fatigue).
- Not including important event details in the notification.

9. Production Best Practices

- Start with high-risk events and add more over time.
- Use clear and specific event patterns.
- Include relevant event details in the notification.
- Test the alert end-to-end.
- Assign someone to respond when an alert is received.
- Integrate with incident management tools (e.g. Slack, PagerDuty).
- Review and update your rules regularly.

10. Key Takeaways

- EventBridge routes AWS events to targets based on rules.
- SNS delivers notifications to the right people.
- Test your alerts to ensure they work.
- Focus on high-risk events and actionable alerts.
- A notification is only valuable if someone investigates and responds.
- Regularly review and improve your alert rules.

17 Respond to a Suspected Compromised Credential

VERIQTA

1. Key Concepts

Compromised Credential
An AWS access key, secret key or other credential that may have been exposed to an unauthorized person.

Containment
Take immediate steps to limit further damage.

Credential Revocation
Deactivate or delete the compromised credential.

Evidence Preservation
Collect and preserve logs and evidence for investigation before making major changes.

Incident Escalation
Notify the right people (security team, management, cloud provider if required).

2. Incident Response Flow

1. **Detect / Suspect Compromise** (e.g. exposed access key)
2. **Investigate** (Check CloudTrail, recent activity, source IPs, used services)
3. **Contain** (restrict permissions and deactivate the credential)
4. **Replace** (create new credential securely and update applications)
5. **Review and Escalate** (confirm no other changes, document findings, and notify relevant teams)

3. What to Investigate

When was the credential used?
(Check CloudTrail for the first and last usage time.)

Where was it used from?
(Review source IP addresses and locations.)

What services and actions were accessed?
(e.g. EC2, S3, IAM, RDS, etc.)

What resources were created, updated or deleted?
(Check for unauthorized changes across your account.)

Was the credential used to create new credentials?
(Additional IAM users, access keys or roles.)

4. Practical: Investigate the Credential

1. Open CloudTrail in the AWS console.
2. Search for events using the compromised access key or IAM user.
3. Review the event history (time, source IP, userAgent, services and actions).
4. Identify any suspicious or unauthorized activity.
5. Document your findings.

AWS CLI (example: lookup access key usage)

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=AccessKeyId,
AttributeValue=AKIAEXAMPLE123 \
  --max-results 50
```

5. Revoke the Compromised Credential

1. In IAM, locate the affected user or access key.
2. Deactivate the access key immediately (or delete it if no longer needed).
3. Remove any unnecessary permissions.
4. If the IAM user is no longer needed, disable or delete the user.

AWS CLI (example: deactivate access key)

```
aws iam update-access-key \
  --user-name <username> \
  --access-key-id AKIAEXAMPLE123
  --status Inactive
```

Note: Deactivating the key stops further use, but you must still investigate what the attacker may have done.

6. Replace the Credential Safely

1. Create a new access key for the user (if still needed).
2. Store the new key securely (e.g. IAM Secrets Manager, environment variables).
3. Update all applications and systems that use the key.
4. Test that everything works with the new credential.
5. Follow the principle of least privilege.

AWS CLI (example: create new access key)

```
aws iam create-access-key \
  --user-name <username>
```

7. Check for Wider Impact

- Look for changes to other resources (EC2, S3, RDS, IAM, etc.).
- Check for new IAM users, roles or policies that were created.
- Review CloudTrail and AWS Config for configuration changes.
- Verify there is no ongoing unauthorized access from unfamiliar IP addresses.

8. Incident Escalation and Documentation

- Notify your security team, management and relevant stakeholders.
- Document the timeline, findings, actions taken and remaining risks.
- If required by your organization, escalate to AWS Support.
- Keep evidence (logs, screenshots, reports) in a secure location.

9. Common Mistakes

- Only deleting the access key without investigating the activity.
- Failing to check CloudTrail for what the attacker did.
- Not reviewing for new IAM users, roles or policies.
- Assuming there was no damage because the key is now deleted.
- Not updating applications that still use the old key.
- Not informing the security team or management.

10. Key Takeaways

- Respond quickly to suspected compromised credentials.
- Contain the damage by revoking or deactivating the credential.
- Investigate the activity using CloudTrail and other AWS logs.
- Replace the credential securely and follow least privilege.
- Deleting a compromised credential does not prove that the attacker made no other changes.
- Always document your findings and escalate according to your organization's process.

18 Protect Data Against Deletion and Recovery Failure

VERIQTA

1. Key Concepts

S3 Versioning
Keeps multiple versions of an object so you can recover previous versions after accidental deletion or changes.

Backup Recovery Points
Restore data from previous backups when data is lost or corrupted.

AWS Backup
A managed service to create and manage backups across AWS services (e.g. S3, EBS, RDS, etc.).

Encryption vs Recoverability
Encryption protects data from unauthorized access, but it does not prevent deletion or guarantee recovery.

2. How S3 Versioning Works



Original object (version 1) → Modified object (version 2) → Deleted object (marked, not permanently lost) → Restore previous version (version 1)

S3 Version History (example)

Version ID	Last modified	Size	Status
3HL4kq...Q1	2026-10-30 10:15	1.2 KB	Current version
9Jd8mP...2K	2026-10-30 09:50	1.2 KB	Previous version
7Fg2nT...8X	2026-10-29 14:22	1.1 KB	Previous version
1A9bCq...5Z	2026-10-28 11:03	1.0 KB	Previous version

3. AWS Backup Overview

Centralized Backup
Create and manage backups for multiple AWS services in one place.

Backup Plans
Define schedules, retention periods and lifecycle rules.

Recovery Points
Each backup creates a recovery point that you can restore from.

Supports Multiple Services
Works with S3, EBS, RDS, DynamoDB, EC2 and more.

Automated and Compliant
Helps you meet business and regulatory requirements for data protection.

4. Practical: Enable S3 Versioning

1. Go to the S3 console and create a test bucket.
2. Open the bucket properties and enable Versioning.
3. Upload a test file (version 1).
4. Modify the file and upload again (version 2).
5. Delete the file (it will be marked deleted, not permanently removed).
6. View the object versions and restore the earlier version.

5. Verify and Restore a Previous Version

1. Go to the bucket and open the object.
2. Click on the Versions tab.
3. Identify the version you want to restore.
4. Download or copy the previous version.
5. Confirm that the content is restored correctly.

Object versions (example)

Version ID	Last modified	Size	Actions
3HL4kq...Q1	2026-10-30 10:15	1.2 KB	✓ Current
9Jd8mP...2K	2026-10-30 09:50	1.2 KB	Restore
7Fg2nT...8X	2026-10-29 14:22	1.1 KB	Restore
1A9bCq...5Z	2026-10-28 11:03	1.0 KB	Restore

6. AWS CLI Example Commands

```
# Enable versioning on a bucket
aws s3api put-bucket-versioning \
    --bucket my-test-bucket \
    --versioning-configuration \
     Status=Enabled
```

```
# Upload a test file
echo "version 1" > file.txt
aws s3 cp file.txt s3://my-test-bucket/
```

```
# Modify and upload again
echo "version 2" > file.txt
aws s3 cp file.txt s3://my-test-bucket/
```

```
# List object versions
aws s3api list-object-versions \
    --bucket my-test-bucket \
    --prefix file.txt
```

```
# Restore a previous versions
aws s3api copy-object \
    --bucket my-test-bucket \
    --copy-source my-test-bucket/file.txt?versionId=9Jd8mP...2K \
    --key file.txt
```

7. Key Differences

Feature	Encryption	Versioning / Backup
Purpose	Protects data from unauthorized access	Protects against accidental deletion and helps with recovery
Prevents deletion?	✗ No	✓ Yes (versioning) / Yes (backups)
Helps with recovery?	✗ No	✓ Yes
What it does	Encrypts data at rest or in transit	Keeps multiple versions or restores from backups

8. Common Beginner Mistakes

- ✗ Assuming encrypted data is protected against accidental deletion.
- ✗ Not enabling versioning on important S3 buckets.
- ✗ Not testing the restore process before an actual incident.
- ✗ Keeping backup retention too short.
- ✗ Relying only on delete protection without backups.
- ✗ Forgetting that versioning does not protect against bucket deletion (it must be combined with other controls).

9. Production Best Practices

- ✓ Enable S3 versioning on all important buckets.
- ✓ Use AWS Backup for a broader backup strategy across multiple services.
- ✓ Set appropriate retention periods based on your business and compliance needs.
- ✓ Regularly test your restore process.
- ✓ Use lifecycle policies to manage old versions and control storage costs.
- ✓ Combine versioning, backups, and access controls for stronger data protection.

10. Key Takeaways

- ✓ S3 versioning keeps previous versions of your objects.
- ✓ AWS Backup provides recovery points for multiple services.
- ✓ Encryption protects data, but it does not prevent deletion.
- ✓ Always test your recovery process before you need it.
- ✓ Use retention policies and lifecycle rules.
- ✓ Data protection requires multiple controls working together.

19 Audit Your AWS Environment Before Launch

VERIQTA

1. Purpose and Benefits

- Find and fix security gaps before your environment goes live.
- Reduce the risk of data exposure, unauthorized access and service disruption.
- Ensure your environment meets security best practices and compliance requirements.
- Build a repeatable security review process for every new project or environment.

2. What to Review

- **Root Access** (No active use, MFA enabled)
- **IAM Permissions** (least privilege, no unused access)
- **Credentials** (no hardcoded keys, no exposure)
- **Network Rules** (security groups, NACLs, open ports)
- **S3 Public Access** (block public access, no public buckets)
- **Encryption** (at rest and in transit)
- **CloudTrail** (enabled, logging to secure bucket)
- **GuardDuty** (enabled and no critical findings)
- **Alerts and Monitoring** (EventBridge, SNS, log alarms)
- **Recovery Readiness** (backups, versioning, tested restore)

3. Review Approach

1. Use the checklist to review each area in your AWS environment.
2. Collect evidence (screenshots, CLI output or configuration).
3. Identify any issues or misconfigurations.
4. Assess the severity and business impact.
5. Assign an owner and target remediation date.
6. Recheck after remediation to confirm the issue is fixed.

4. Practical: Run Security Checks

1. Check if root account has MFA enabled.
2. Review IAM users, roles and permissions.
3. Look for exposed or unused access keys.
4. Check security group and NACL rules for unnecessary open access.
5. Verify S3 public access settings.
6. Confirm encryption for S3, EBS and RDS.
7. Check that CloudTrail is enabled and logs are stored securely.
8. Review GuardDuty findings.
9. Verify alerts are configured and working.
10. Confirm backup and recovery readiness.

5. Example Security Assessment

Area	Finding	Evidence	Severity	Owner	Status
Root Access	MFA not enabled for root account	Console screenshot	Critical	Cloud Team	Open
IAM Permissions	User with AdministratorAccess	IAM policy	High	Cloud Team	In Progress
Credentials	Unused access key older than 90 days	IAM credentials report	Medium	Security Team	Open
Network Rules	SSH open to 0.0.0.0/0	Security group configuration	High	Infrastructure	Resolved
S3 Public Access	Public bucket found	S3 bucket policy	Critical	Cloud Team	Open
Encryption	EBS volume not encrypted	EC2 instance details	Medium	Infrastructure	In Progress
CloudTrail	Not enabled in region	AWS console	High	Cloud Team	Open
GuardDuty	Critical finding detected	GuardDuty console	High	Security Team	In Progress
Alerts	No SNS notification configured	EventBridge rule	Medium	Cloud Team	Open
Recovery	No backups configured for RDS	AWS Backup console	High	Infrastructure	Open

6. Useful AWS CLI Commands (Examples)

```
# Check root account MFA status
aws iam get-account-summary
# List IAM users
aws iam list-users
# Check access keys for a user
aws iam list-access-keys --user-name <username>
# Check S3 public access settings
aws s3api get-public-access-block --bucket <bucket-name>
# Check security group rules
aws ec2 describe-security-groups --group-ids <sg-id>
# Check if CloudTrail is enabled
aws cloudtrail describe-trails
# Check GuardDuty findings
aws guardduty list-findings --max-results 50
```

7. Common Issues to Look For

- Root account without MFA.
- Overly permissive IAM policies.
- Exposed or unused access keys.
- Security groups open to the internet (e.g. SSH 0.0.0.0/0).
- Public S3 buckets or missing block public access.
- Unencrypted data (S3, EBS, RDS).
- CloudTrail not enabled or logs stored in the same account.
- GuardDuty disabled or critical findings not investigated.
- Missing alerts for important events.
- No backups or untested recovery process.

8. Production Best Practices

- Always enable MFA for the root account and all users.
- Use least privilege IAM permissions.
- Rotate and monitor access keys regularly.
- Restrict network access to only what is needed.
- Block public access on all S3 buckets.
- Encrypt data at rest and in transit.
- Enable CloudTrail in all regions and store logs in a dedicated, secure account.
- Keep GuardDuty enabled and investigate findings.
- Configure alerts for critical security events.
- Implement regular backups and test recovery.

9. Key Takeaways

- A security review before launch prevents serious problems.
- Use a checklist and collect evidence for every finding.
- Fix issues based on severity and business impact.
- Regularly repeat this review for new and existing environments.
- Security is an ongoing process, not a one-time task.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

20 Final Lab: Prove Your AWS Environment Is Secure

VERIQTA

1. Scenario Overview

The application is scheduled for launch, but a final review reveals security issues that must be fixed before going live.

Your Goal

Identify the issues, explain the risk, apply the approved fixes, and prove that the environment is secure and ready for launch.

Deliverable

A junior engineer's AWS Security Readiness Report with findings, evidence, remediation steps and any remaining risks.

2. Issues Discovered in the Review

1. Excessive Permissions

Some IAM users or roles have more permissions than required (e.g. AdministratorAccess).

2. Exposed Network Rule

A security group allows unrestricted inbound access (e.g. 0.0.0.0/0 on SSH or unnecessary open ports).

3. Incomplete Monitoring

CloudTrail, GuardDuty or alerts are not fully configured or not sending notifications.

Additional Areas to Check

S3 public access, encryption, backup and recovery, root access and MFA, and other common security gaps.

3. Lab Objectives

1. Find and document all security issues in the environment.
2. Explain the risk and potential business impact for each issue.
3. Apply the approved fixes using AWS best practices.
4. Collect verification evidence to prove the issues are resolved.
5. Identify any remaining risks and document them.
6. Produce a clear and professional Security Readiness Report.

4. Practical Tasks

1. Review IAM users, roles and policies for excessive permissions.
2. Inspect security groups and network rules for unnecessary open access.
3. Check S3 buckets for public access and encryption.
4. Verify CloudTrail is enabled and logs are stored securely.
5. Confirm GuardDuty is enabled and review recent findings.
6. Check that alerts (EventBridge + SNS) are working.
7. Verify AWS Backup and recovery readiness.
8. Apply the necessary fixes.
9. Recheck and confirm that the issues are resolved.

5. Example Findings and Fixes

Area	Finding	Risk	Fix Applied
IAM	User with AdministratorAccess	Full access if compromised	Replaced with least privilege policy
Security Group	SSH open to 0.0.0.0/0	Unauthorized access from internet	Restricted to specific IP or VPN
S3 Bucket	Public access enabled	Data exposure	Blocked public access and enabled encryption
CloudTrail	Not enabled	No audit trail of API activity	Enabled and sent logs to S3
GuardDuty	Not enabled	Threats not detected	Enabled and reviewed findings
Alerts	No notifications	Missed security events	Configured EventBridge rule and SNS alerts
Backup	No backup plan	Data loss and long recovery time	Created AWS Backup plan with retention

6. Useful AWS CLI Commands (Examples)

```
# List IAM users and attached policies
aws iam list-users
aws iam list-attached-user-policiee --user-name <username>
# Check security group rules
aws ec2 describe-security-groups \
  --group-ids <sg-id>
# Check S3 bucket public access
aws s3api get-public-access-block --bucket <bucket-name>
# Check CloudTrail status
aws cloudtrail describe-trails
# Check GuardDuty status
aws guardduty list-detectors
# Check AWS Backup plans
aws backup list-backup-plans
```

7. Report Template (Example)

1. **1.Executive Summary**
   Brief overview of the environment and readiness status.
2. **2.Findings Table**
   Issue, risk, evidence, severity, owner, status.
3. **3.Remediation Actions**
   Steps taken to fix each issue.
4. **4.Verification Evidence**
   CLI output, screenshots or configuration details showing the fix.
5. **5.Remaining Risks**
   Any risks that could not be fully resolved.
6. **6.Conclusion**
   Is the environment ready for launch?

8. Grading Criteria (Self-Check)

- All required areas were reviewed.
- Security issues were correctly identified.
- Risks were clearly explained.
- Approved fixes were applied using best practices.
- Verification evidence was collected.
- Remaining risks were documented.
- Report is clear, structured and professional.

9. Key Takeaways

- A final security review prevents costly incidents.
- Least privilege, secure network rules and proper monitoring are essential before launch.
- Fixing issues is not enough, you must prove the fix with evidence.
- A good security report shows that the environment is ready and that you understand the risks.